

Google Search Trends

DATA SCIENCE REPORT



by Kadir Arslan

Created: 15.12.2024

Last Edited: 13.03.2026

Google Trends Report

In this report, we will explore the data for specific cases in terms of their popularity in Google Search Trends.

We will examine the stock prices of Tesla and Bitcoin and investigate whether there is a correlation between their prices and their popularity among Google users.

Next, we will investigate the relationship between unemployment rates, and the frequency of how much and how often people search for unemployment benefits on Google.

These Analysis will help us understand the how can someone assess a situation in real world by inspecting Google Search Trends.

Since the data for Google search Trends for any phrase are publicly available, the statistical analysis of their impact upon real-world problems can be significantly beneficial.

Using data science methods, we will explore and address a range of questions, including the following:

How can Google Trends be combined with other Time Series Data ?

- What can the popularity of search terms tell us about the world?
- How and how strong can the popularity of Google Search terms reveal information on real Data ?
- How does the search volume for "Bitcoin" relates to the price of Bitcoin?
- How can search volume for a hot stock like Telsa relate to that stock's price ?
- How searches for "Unemployment Benefits" in Google vary with the actual unemployment rate in the United States ?
- Can someone estimate the real data or tendencies of values in future with using data from Google Search Trends?

Source Data Files (CSV Format)

Tesla Search Trend vs Price: [github.com](#)

Bitcoin Search Trend: [github.com](#)

Daily Bitcoin Price: [github.com](#)

Unemployment Search Trends vs unemployment rate (2004-2019): [github.com](#)

Unemployment Search Trends vs unemployment rate (2004-2020): [github.com](#)

- For “Unemployment” the abbreviation “EU” will be used in this report.

Imported Libraries:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import register_matplotlib_converters
register_matplotlib_converters()
import matplotlib.dates as mdates
```

Analysis for Tesla

Getting Initial Insights into Data

First, we are going to examine “Tesla” data.

To get a basic understanding of many of the properties of a DataFrame, there are several useful Pandas functions available to us:

shape(): It returns number of columns and rows of a dataset.

head(n): It returns the first n rows of a dataset. (default n=5)

info(): It provides a concise summary of a DataFrame, including the number of entries, column names, data types, and non-null values.

describe(): It generates a statistical summary of the DataFrame, showing measures such as count, mean, standard deviation, minimum, maximum, and quartiles for numerical columns.

```
df_tesla = pd.read_csv('csv_files/TESLA_Search_Trend_vs_Price.csv')
print("\nTesla Dataframe:")

print("\nShape of the DataFrame:")
print(df_tesla.shape)

print("\nInitial Insight into the DataFrame:")
print(df_tesla.head())

print("\nGeneral Structure of the DataFrame:")
print(df_tesla.info())

print("\nKey Statistics in the DataFrame:")
print(df_tesla.describe())
```

Shape of the DataFrame :

```
(124, 3)
```

The DataFrame of Tesla file has 124 rows 3 columns.

Initial Insight into the DataFrame:

	MONTH	TSLA_WEB_SEARCH	TSLA_USD_CLOSE
0	2010-06-01	3	4.766
1	2010-07-01	3	3.988
2	2010-08-01	2	3.896
3	2010-09-01	2	4.082
4	2010-10-01	2	4.368

The output shows that the dataset contains three columns:

“Month” displays the data for each month. Looking at the 124 rows, it’s clear that we have data spanning nearly ten years. In the second column, we see the Tesla web search coefficients, which use data from Google Trends to show how popular Tesla was that month. The third column represents the monthly closing price of Tesla stock at the market.

What does Google Search Trend Values Mean ?

We can see from our DataFrames that Google's search interest ranges between 0 and 100.

The figures reflect the relative search interest for each region and time period. A value of 100 indicates the highest popularity of the term. A value of 50 means that the term is only half as popular. A value of 0 means that there was insufficient data available for this term.

Basically, Google only provides a scaled number. In short, Google Trends doesn’t exactly specify how many search inquiries occurred for the particular topic, but it gives a nice indication.

General Structure of the DataFrame:

```
<class 'pandas.DataFrame'>
RangeIndex: 124 entries, 0 to 123
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
#   Column          Non-Null Count  Dtype
```

```
0    MONTH          124 non-null    str
1    TSLA_WEB_SEARCH 124 non-null    int64
2    TSLA_USD_CLOSE  124 non-null    float64
dtypes: float64(1), int64(1), str(1)
memory usage: 3.0 KB
None
```

Again, this function also demonstrates that we have 124 rows and 3 columns.

It points out that there are not any NA (not a number) values on the dataset.

We see that the datatype for month are strings. (This is important, we would have to convert to DateTime objects to be able to compare the values.)

Lastly, google trend values are integer(0-100) and prices are held on float datatype.

Key Statistics in the DataFrame:

	TSLA_WEB_SEARCH	TSLA_USD_CLOSE
count	124.000000	124.000000
mean	8.725806	50.962145
std	5.870332	65.908389
min	2.000000	3.896000
25%	3.750000	7.352500
50%	8.000000	44.653000
75%	12.000000	58.991999
max	31.000000	498.320007

Describe() function returns descriptive statistics for the variables in the DataFrame.

It displays the number of columns. It shows the minimum, maximum, and mean values for those specific columns. Finally, this function allows us to retrieve important statistical measures of the data, such as the standard deviation (std) and the quartiles of the columns. It did not return any data on "Month" since it was held on "string".

The statistics demonstrate that,

The average closing price of Tesla stock is approximately 50.96 USD, with values ranging from 3.90 USD to 498.32 USD, indicating a high variability over the observed period.

Similarly, the web search values show noticeable variation, with a mean of 8.73 and a maximum value of 31, suggesting fluctuating public interest over time.

Analysis for Bitcoin

Bitcoin is a decentralized digital currency introduced in 2009 that allows people to send and receive money over the internet without the need for a bank or central authority.

Here we are going to examine if there is a connection pattern between actual Bitcoin data which includes prices, ups and downs and Google search Trends for Bitcoin by users.

We have two different CSV files for the Bitcoin Analysis: Price and Google Trends

DataFrame for Bitcoin Google search trends:

```
df_btc_search = pd.read_csv('csv_files/Bitcoin_Search_Trend.csv')

print("\nBitcoin Search Dataframe:")

print("\nShape of the DataFrame:")
print(df_btc_search.shape)

print("\nInitial Insight into the DataFrame:")
print(df_btc_search.head())
```

Shape of the DataFrame:

```
(73, 2)
```

Initial Insight into the DataFrame:

	MONTH	BTC_NEWS_SEARCH
0	2014-09	5
1	2014-10	4
2	2014-11	4
3	2014-12	4
4	2015-01	5

The results show that the DataFrame containing the Google Trends data for Bitcoin consists of 73 rows and 2 columns.

The DataFrame displays the corresponding Google Trends coefficients for Bitcoin on a monthly basis.

DataFrame for Bitcoin prices:

```
df_btc_price = pd.read_csv('csv_files/Daily_Bitcoin_Price.csv')
print("\nBitcoin Price Dataframe:")

print("\nShape of the DataFrame:")
print(df_btc_price.shape)

print("\nInitial Insight into the DataFrame:")
print(df_btc_price.head())
```

Shape of the DataFrame:

(2204, 3)

Initial Insight into the DataFrame:

	DATE	CLOSE	VOLUME
0	2014-09-17	457.334015	21056800.0
1	2014-09-18	424.440002	34483200.0
2	2014-09-19	394.795990	37919700.0
3	2014-09-20	408.903992	36863600.0
4	2014-09-21	398.821014	26580100.0

The data shows that the Bitcoin Price DataFrame has a significantly larger volume with 2204 rows and three columns.

The Data demonstrates Closing price of Bitcoin stock and its volume at the stock market on Daily Basis.

While the DataFrame for Google trends held on “monthly” the DataFrame for prices is set to a “Daily” Basis. We will take care of this later on.

Analysis for Unemployment in the United States of America

The unemployment rate in the United States is an important economic indicator that not only reflects the economic conditions but also influences people’s behavior, as higher unemployment often leads individuals to seek unemployment benefits, financial assistance, and new job opportunities more frequently.

In this analysis, we are going to explore the relationship between unemployment rates and the popularity of Google inquiries for unemployment benefits.

DataFrame for Unemployment in the USA:

```
df_unemployment =  
    pd.read_csv('csv_files/UE_Benefits_Search_vs_UE_Rate_2004-19.csv')  
  
print("\nUnemployment Dataframe:")  
  
print("\nShape of the DataFrame:")  
print(df_unemployment.shape)  
  
print("\nInitial Insight into the DataFrame:")  
print(df_unemployment.head())
```

Shape of the DataFrame:

```
(181, 3)
```

Initial Insight into the DataFrame:

	MONTH	UE_BENEFITS_WEB_SEARCH	UNRATE
0	2004-01	34	5.7
1	2004-02	33	5.6
2	2004-03	25	5.8
3	2004-04	29	5.6
4	2004-05	23	5.6

The unemployment DataFrame has 181 rows and 3 columns. Similar with Tesla data, we have monthly data from 2004 onwards, organized in rows.

At further two data columns, it demonstrates the frequency of Google searches related to unemployment, as well as the unemployment rates in the United States for each month from 2004 to 2019.

Investigation of DataFrames for Missing Values

To determine if a DataFrame has missing values (na), we use the functions `.isna()`, `.values()` and `any()` together.

```
print(f'Missing values for Tesla?: {df_tesla.isna().values.any()}')
print(f'Missing values for U/E?: {df_unemployment.isna().values.any()}')
print(f'Missing values for BTC Search?: {df_btc_search.isna().values.any()}')
print(f'Missing values for BTC price?: {df_btc_price.isna().values.any()}')
```

output:

```
Missing values for Tesla?: False
Missing values for U/E?: False
Missing values for BTC Search?: False
Missing values for BTC price?: True
```

We have missing values just for the DataFrame of Bitcoin prices.

Let's check if they are significant for the Analysis:

```
print(f'Sum of missing values for BTC price?: {df_btc_price.isna().values.sum()}')
>> Sum of missing values for BTC price?: 2
```

There are just two rows in the DataFrame, so it is okay to remove them from the data.

Removing Entries with Missing Values

To eliminate the NaN entries, we use the function ***.dropna()*** :

```
df_btc_price = df_btc_price.dropna()
```

Now, we don't have any NaN values in our DataFrames.

Converting strings to DateTime Objects in a DataFrame

During investigating Tesla DataFrame, with the help of the function ***.info()***, we have seen that the column which shows values for “months” are originated as “strings”.

This is a problem because we want to compare the data across time periods and draw conclusions from them.

We are going to convert the relevant columns at these DataFrames from strings to DateTime to enable comparison and analysis in time intervals.

```
df_tesla.MONTH = pd.to_datetime(df_tesla.MONTH)
df_btc_search.MONTH = pd.to_datetime(df_btc_search.MONTH)
df_unemployment.MONTH = pd.to_datetime(df_unemployment.MONTH)
df_btc_price.DATE = pd.to_datetime(df_btc_price.DATE)
```

Resampling a Daily Data to Monthly Frequency

While all other time variables are recorded at a monthly frequency, the time values in the Bitcoin price DataFrame are recorded on a daily basis.

We are going to aggregate the daily values into corresponding monthly values by calculating the average of the remaining data for each month.

```
df_btc_monthly = df_btc_price.resample('ME', on='DATE').mean()
```

Now, we have also monthly data for the Bitcoin prices:

```
print(df_btc_monthly.shape)
print(df_btc_monthly.head())
```

new data:

```
(73, 2)
      DATE      CLOSE      VOLUME
2014-09-30  407.182428  2.934864e+07
2014-10-31  364.148873  2.912885e+07
2014-11-30  366.099799  2.199111e+07
2014-12-31  341.267871  1.784201e+07
2015-01-31  248.782547  3.544555e+07
```

The Shape of the DataFrame for Bitcoin price was (2204, 3).

The column “Date” is now turned into index of the DataFrame as datetime objects.

2204 daily “date” values were converted into 73 monthly values.

All of these four DataFrames are now ready to generate graphs and perform analyses upon them.

Defining Locators and Formatters

In this step, we will define date locators and formatters for the plot axis:

```
years = mdates.YearLocator()
months = mdates.MonthLocator()
years_fmt = mdates.DateFormatter('%Y')
```

Adding Locator Tick Marks:

When working with time series, it is often quite difficult to display the tick marks in charts exactly as you would like. That is why the Locator helper functions exist.

The year and month locators are used to control the placement of ticks on the time axis, while the date formatter specifies that the labels should display the year in numeric format.

Generating Graphs

Comparison between Search Trends and Tesla stock Prices

In this analysis we are going to examine the relationship between public interest and market valuation by exploring the correlation between Google search trends for “Tesla” and the company’s performance at stock market from 2011 to 2020.

By comparing the popularity of search queries with actual market trends, we aim to determine to what extent general investor interest serves as a reliable indicator of Tesla’s stock price performance.

Creating Dual-Axis Line Plot

Code:

```
sns.set_theme(style="whitegrid", context="talk", font_scale=0.8)

# Create figure with high resolution
plt.figure(figsize=(10, 5), dpi=150)
plt.title('Tesla Web Search Trend vs. Stock Price', fontsize=14,
fontweight='bold', pad=15)

ax1 = plt.gca()
ax2 = ax1.twinx()

# Plot using Seaborn
sns.lineplot(
    data=df_tesla,
    x='MONTH',
    y='TSLA_USD_CLOSE',
```

```

        color='#E6232E',
        linewidth=2.5,
        ax=ax1,
        label='TESLA Stock Price'
    )

sns.lineplot(
    data=df_tesla,
    x='MONTH',
    y='TSLA_WEB_SEARCH',
    color='#2B90B6',
    linewidth=2.5,
    ax=ax2,
    label='Search Trend'
)

# Axis Labels
ax1.set_xlabel('Date', fontsize=11, fontweight='bold', color='gray')
ax1.set_ylabel('TESLA Stock Price (USD)', color='#E6232E', fontsize=11,
fontweight='bold')
ax2.set_ylabel('Search Trend Volume', color='#2B90B6', fontsize=11,
fontweight='bold')

# Setting Limits
# We are applying this just to get rid of small empty spaces at the start and end
of the graph.
ax1.set_ylim([0, 600])
ax1.set_xlim([df_tesla.MONTH.min(), df_tesla.MONTH.max()])

# Making gridlines very transparent.
# When working with two axes, the gridlines for the second should be turned off.
ax1.grid(True, alpha=0.2)
ax2.grid(False)

# Formating the x-axis dates
ax1.xaxis.set_major_locator(years)
ax1.xaxis.set_major_formatter(years_fmt)
ax1.xaxis.set_minor_locator(months)
plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha='right')

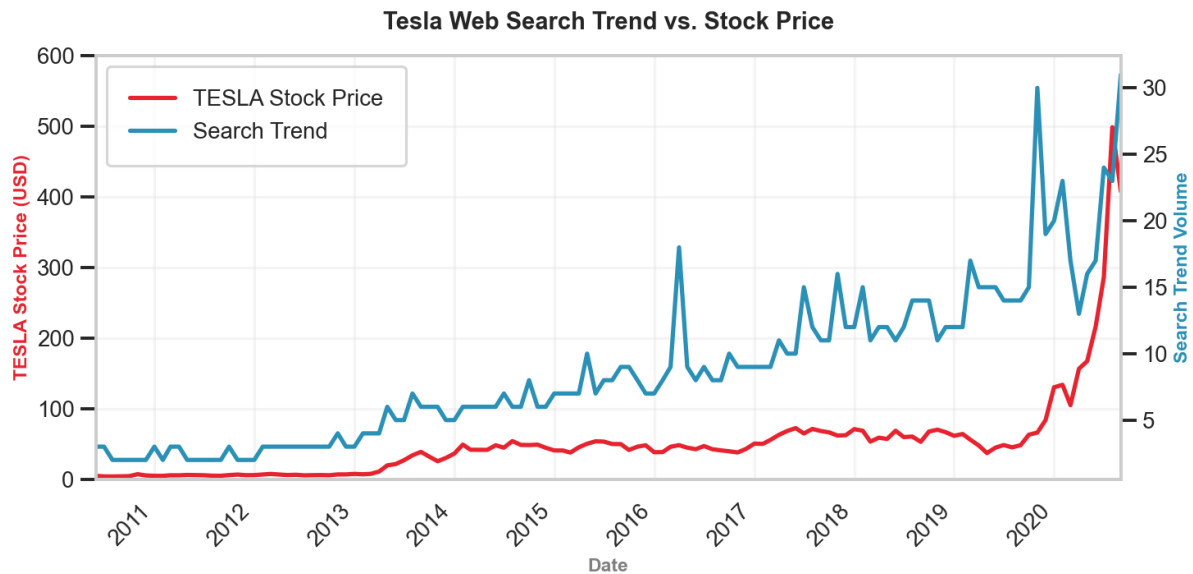
# Combining the Legends into one clean box
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left', frameon=True,
shadow=False, borderpad=1)
ax2.get_legend().remove() # Remove the redundant secondary Legend

# Adjusting layout so that nothing gets cut off.
plt.tight_layout()

plt.show()

```

Graph:



The graph shows a coherent pattern for both topics that is accompanied by an upward trend.

Given the fact that there are no significant ways to determine the exact relationship between two dates, it seems that there are some special points that may indicate reaction situations in the analysis.

The figure points out that there is spike of interest in the middle of the chart in March 2016. This was when the Tesla Model 3 was unveiled.

It also indicates a spike in November 2019, which was the that Tesla unveiled Cybertruck in Los Angeles. That was one of the most significant and attention-grabbing events in Tesla's history.

Also, we can clearly see that the most recent spikes in search coincide, not with the release of a new car, but the roaring stock price for the company.

The steady rise in public interest in Tesla between 2016 and 2020 may have served as an indicator of the company's enormous success after 2020.

Comparison between Search Trends and Prices of Bitcoin

Cryptocurrency markets are heavily influenced by public opinions, which is why the relationship between financial performance and public attention can present a fascinating area for data analysis.

In this analysis we are going to investigate the statistical correlation between Bitcoin's market developments and Google Search trends to determine if search volume acts as a reliable indicator for price fluctuations.

Creating Dual-Axis Line Plot

Code:

```
# Plot configuration for bitcoin

sns.set_theme(style="ticks", context="talk")

# Creating the figure and the primary axis
fig, ax1 = plt.subplots(figsize=(10, 5), dpi=120)

# Defining distinct colors for two Variables
color_price = '#2ca02c'
color_search = '#1f77b4'

# Plotting stock price on the primary y-axis
ax1.set_xlabel('Date', fontweight='bold', fontsize=12)
ax1.set_ylabel('BTC Price (USD)', color=color_price, fontweight='bold',
              fontsize=12)

# Set the minimum and maximum values on the axes for a better view
ax1.set_ylim(bottom=0, top=15000)
ax1.set_xlim([df_btc_monthly.index.min(), df_btc_monthly.index.max()])

# Plotting the line for the price data
sns.lineplot(data=df_btc_monthly, x=df_btc_monthly.index, y='CLOSE',
            color=color_price, linewidth=2.5, ax=ax1)

# Color match the ticks to the line and add a soft grid
ax1.tick_params(axis='y', labelcolor=color_price)
ax1.grid(True, axis='y', linestyle='--', alpha=0.4)

# Creating the secondary (right) y-axis sharing the same x-axis
ax2 = ax1.twinx()
```



```
# Plotting Google Search Trend of Bitcoin on the secondary y-axis
ax2.set_ylabel('Search Trend', color=color_search, fontweight='bold', fontsize=12)

# Plotting the line for the data
sns.lineplot(data=df_btc_search, x=df_btc_monthly.index, y='BTC_NEWS_SEARCH',
             color=color_search, linewidth=2.5, ax=ax2)

# Color match the ticks for the second axis
ax2.tick_params(axis='y', labelcolor=color_search)
ax2.set_ylim(bottom=0)

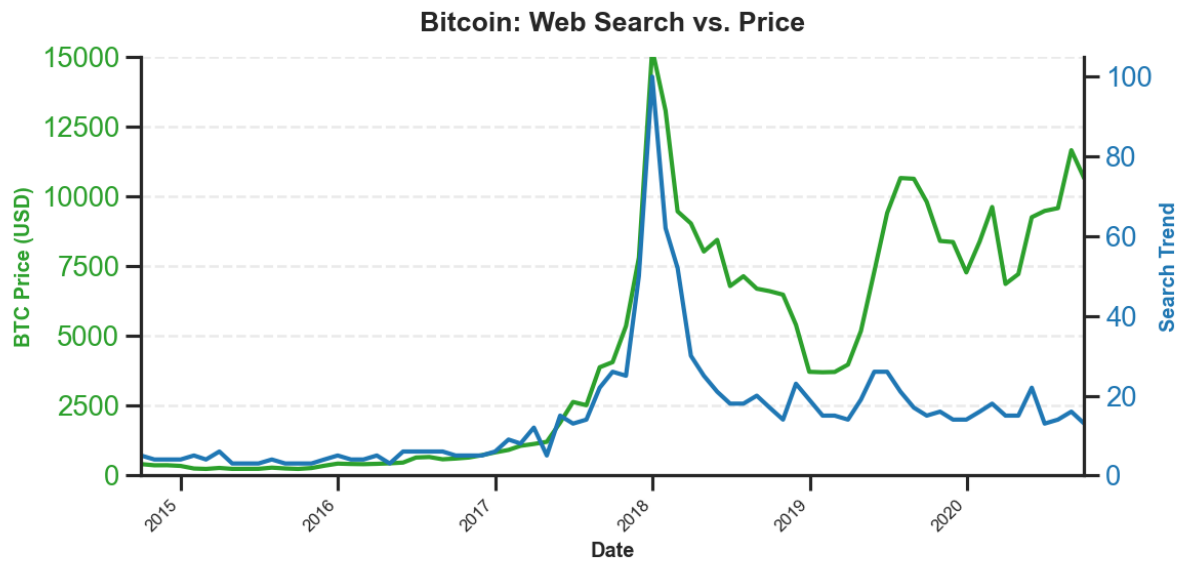
# Cleaning up the x-axis formatting (our original locators)
years = mdates.YearLocator()
years_fmt = mdates.DateFormatter('%Y')
ax1.xaxis.set_major_locator(years)
ax1.xaxis.set_major_formatter(years_fmt)

# Rotating the labels on the primary axis to read them better
plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha='right', fontsize=10)

# Adding title and clean up borders
plt.title('Bitcoin: Web Search vs. Price', fontsize=16, fontweight='bold', pad=15)
sns.despine(top=True, right=False)

fig.tight_layout()
plt.show()
```

Graph:



The chart shows a strong correlation between the sharp price fluctuations in early 2018 and increased search volumes—a pattern that mirrors that of Tesla.

In late 2017 and early 2018, everyone was talking about Bitcoin (and buying it), causing search queries to reach a record high.

Interestingly, there was quite a huge spike in bitcoin prices in Q1 of 2019 but this time the increase in search volume was much less pronounced.

When we investigate this event, we see that on 2nd April, Bitcoin jumped nearly 20% in less than an hour. The reason for that was a massive, algorithmic buy order. A group of coordinated buyers purchased roughly \$100 million worth of Bitcoin spread across the major exchanges.

This particular occurrence highlights that for the areas which are prone to manipulations, analyzing a data just based on public interests is not a good idea.

Looking at the before-and-after images from 2018, the figure suggests that the correlation between fluctuations in public interest and the actual data may weaken once people have become accustomed to this area.

Comparison between Unemployment rates and Search Trends for Unemployment Benefits

In this analysis we are going to examine the relationship between official macroeconomic unemployment rates and the volume of Google search queries for "unemployment benefits."

By correlating these datasets, we aim to determine whether real-time digital search behavior can serve as a reliable, predictive leading indicator for general labor market trends.

Recalling the Data:

```
df_unemployment = pd.read_csv('csv_files/UE_Benefits_Search_vs_UE_Rate_2004-19.csv')
```

```
(181, 3)
```

	MONTH	UE_BENEFITS_WEB_SEARCH	UNRATE
0	2004-01	34	5.7
1	2004-02	33	5.6
2	2004-03	25	5.8
3	2004-04	29	5.6
4	2004-05	23	5.6

Dealing with Continuous Fluctuations

As observed in our initial exploration of the DataFrame, the data exhibits significant month-to-month fluctuations. To reduce this noise and highlight the underlying trends more clearly, we are going to smooth the visualization using the rolling average technique

Taking 6 months average:

```
# Calculate rolling averages
df_unemployment['UNRATE_ROLLING'] =
    df_unemployment['UNRATE'].rolling(window=6).mean()
```

```
df_unemployment['SEARCH_ROLLING'] =  
    df_unemployment['UE_BENEFITS_WEB_SEARCH'].rolling(window=6).mean()
```

We are using `.rolling(window=6).mean()` in pandas to calculate the moving average over the last 6 rows, meaning that for each row, it is going to take the current value and the previous 5 values and returns their average. This method is often used to smooth time series data or reduce short-term fluctuations.

Creating Stacked Subplot Graphs

Here, we will use a stacked subplot graph to break the data down into stacked sub-charts that share the same time frame, so that the data points are displayed more clearly.

Code for the graph:

```
sns.set_theme(style="white", context="talk")  
  
# Create a figure with 2 subplots sharing the x-axis  
fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, figsize=(10, 6), dpi=150,  
                               sharex=True)  
  
# Defining Different colors for the data  
color_ue = '#2b5b84'  
color_search = '#d95f02'  
  
# --- Top Plot: Unemployment Rate ---  
sns.lineplot(data=df_unemployment, x='MONTH', y='UNRATE_ROLLING',  
             ax=ax1, color=color_ue, linewidth=2.5)  
  
ax1.fill_between(df_unemployment.MONTH, df_unemployment.UNRATE_ROLLING,  
                 color=color_ue, alpha=0.1) # Adds a modern light fill  
ax1.set_ylabel('UE Rate (%)', fontweight='bold', color=color_ue, fontsize=12)  
  
ax1.set_title('Unemployment - Macroeconomic & Search Indicators',  
             fontsize=14, fontweight='bold', pad=15, loc='left')  
  
# --- Bottom Plot: Search Trend ---
```

```
sns.lineplot(data=df_unemployment, x='MONTH', y='SEARCH_ROLLING',
             ax=ax2, color=color_search, linewidth=2.5)
ax2.fill_between(df_unemployment.MONTH, df_unemployment.SEARCH_ROLLING,
color=color_search, alpha=0.1)
ax2.set_ylabel('Search Trend', fontweight='bold', color=color_search, fontsize=12)
ax2.set_xlabel('')

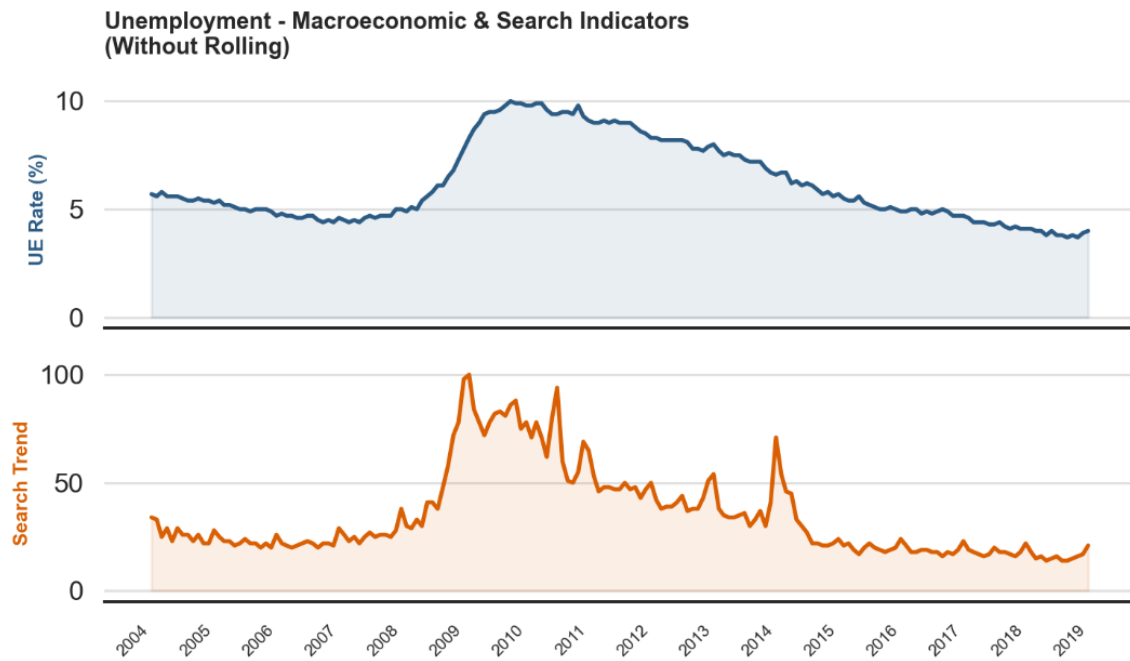
# Formatting and Cleaning up the graph
ax2.xaxis.set_major_locator(mdates.YearLocator())
ax2.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.setp(ax2.xaxis.get_majorticklabels(), rotation=45, ha='right', fontsize=10)

# Adding subtle and light horizontal gridlines
ax1.grid(axis='y', color='lightgrey', linestyle='-', alpha=0.7)
ax2.grid(axis='y', color='lightgrey', linestyle='-', alpha=0.7)

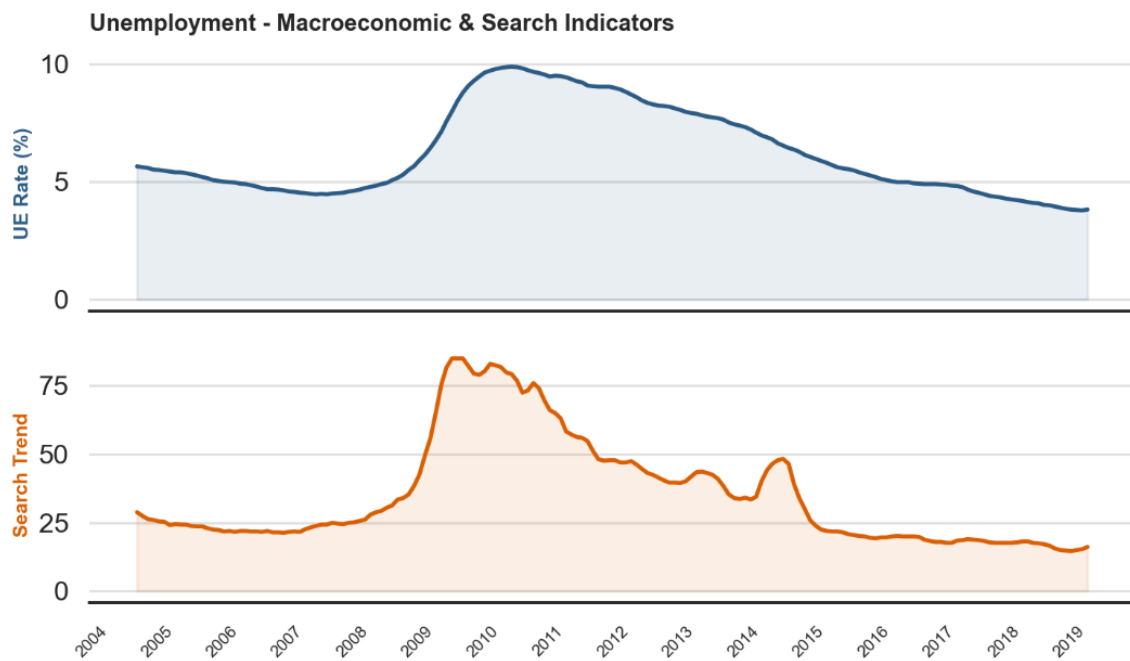
sns.despine(fig, left=True, bottom=False)
plt.tight_layout()
plt.show()
```

Graphs:

Without Rolling Average:



With 6 months Rolling Average:



The first Graph illustrates the unrolled search volume for the term “unemployment benefits,” with vertical dashed lines all around the graph, highlighting periodic peaks. A consistent pattern of increases every December highlights a distinct seasonality in the labor market.

The data also clearly show the impact of the 2008 financial crisis, which triggered a significant escalation in unemployment. A long-term analysis reveals that it took ten years (2007–2017) for the unemployment rate to return to its pre-crisis level.

Throughout this decade, search interest in unemployment benefits remained high but showed a gradual, long-term downward trend that reflected the macroeconomic recovery.

It is noteworthy that, at the end of 2013, an anomaly occurred in which a significant increase in search volume did not translate into a corresponding rise in the actual unemployment rate.

This divergence makes it clear that search trend data is susceptible to external factors or noise and does not always align perfectly with the underlying quantitative metrics.

To mitigate the significant monthly fluctuations in raw search volumes, a 6-month moving average was used in the second chart.

This smoothed graph illustrates a key temporal relationship: An increase in search interest for “unemployment benefits” always precedes a rise in the official unemployment rate. Conversely, search frequency begins to decline even before the unemployment rate starts to fall.

Consequently, these search trends serve as a leading indicator for the economy and provide insight into changes in the official unemployment rate, which traditionally acts as a lagging indicator.

Overall, the smoothed data shows a strong, predictive correlation between search behavior and official macroeconomic indicators.

Unemployment: The Effect of New Data (Covid-19)

While it took nearly a decade for unemployment rates to recover after the 2008 financial crisis, the sudden shock of the 2020 COVID-19 pandemic presents a completely different picture for labor market analysis.

In this analysis, we are going to examine the direct impact of the pandemic on unemployment rates as well as public sentiment, focusing on search trends and concerns related to unemployment benefits.

We have a new data file:

```
df_ue_2020 = pd.read_csv('.../.../UE_Benefits_Search_vs_UE_Rate_2004-20.csv')
```

To Obtain Initial Insight into the DataFrame:

Here, we are going to use `.tail(n)` function to receive last n rows of a data. (default: 5)

```
print("\nDataframe for the Unemployment from 2020:")
print(df_ue_2020.shape)
print(df_ue_2020.tail())
```

Dataframe for the Unemployment from 2020:

```
(200, 3)
```

	MONTH	UE_BENEFITS_WEB_SEARCH	UNRATE
195	2020-04	100	14.7
196	2020-05	63	13.3
197	2020-06	53	11.1
198	2020-07	54	10.2
199	2020-08	50	8.4

Converting the string object to DateTime

```
df_ue_2020.MONTH = pd.to_datetime(df_ue_2020.MONTH)
```


Creating Stacked Subplot Graphs

code:

```
# Plot configuration for bitcoin for eu unemployment -monthly
sns.set_theme(style="whitegrid", context="notebook")

# Create a figure with two subplots sharing the same x-axis
fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, figsize=(10, 6), sharex=True,
                               dpi=120)
fig.subplots_adjust(hspace=0.2) # Bring the plots closer together

# Defining colors for the Graphs
color_unrate = "#7B1FA2"
color_search = "#0288D1"

# Defining the time before covid
before_covid = pd.to_datetime('2019-01')

ax1.set_xlim([before_covid, df_ue_2020['MONTH'].max()])
ax2.set_xlim([before_covid, df_ue_2020['MONTH'].max()])

# Plotting Unemployment Rate
sns.lineplot(x=df_ue_2020['MONTH'], y=df_ue_2020['UNRATE'],
             ax=ax1, color=color_unrate, linewidth=2.5)

# Filling underneath of the line for a subtle-modern look
ax1.fill_between(df_ue_2020['MONTH'], df_ue_2020['UNRATE'], color=color_unrate,
                alpha=0.2)

ax1.set_ylabel('UE Rate (%)', fontsize=10, fontweight='bold', color=color_unrate,
               labelpad=25)
ax1.set_title('US Unemployment Rate vs. Google Search Trends',
              fontsize=14, fontweight='bold', pad=25)

# Plotting Google Search Trends
sns.lineplot(x=df_ue_2020['MONTH'], y=df_ue_2020['UE_BENEFITS_WEB_SEARCH'],
             ax=ax2, color=color_search, linewidth=2.5)

# Again Filling underneath of the line
ax2.fill_between(df_ue_2020['MONTH'], df_ue_2020['UE_BENEFITS_WEB_SEARCH'],
                color=color_search, alpha=0.2)

ax2.set_ylabel('Search Index', fontsize=10, fontweight='bold', color=color_search,
               labelpad=18)
ax2.set_xlabel('Date', fontsize=10, fontweight='bold', labelpad=18)
```

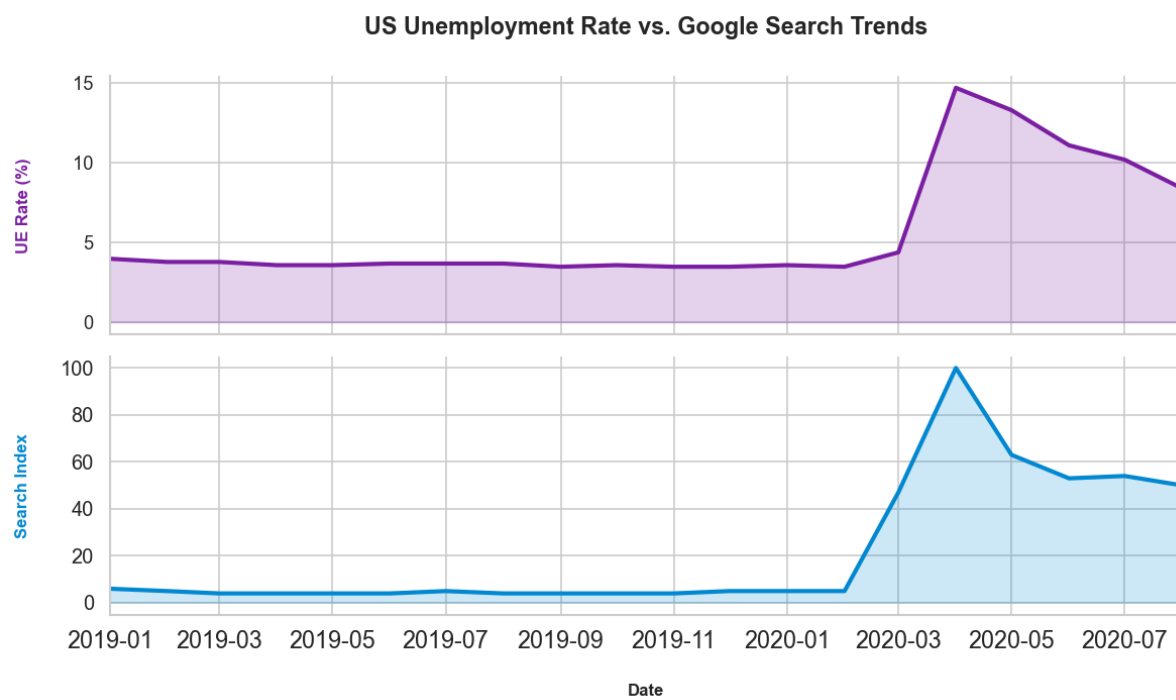
```
# Removing top and right borders for a cleaner look
sns.despine(fig=fig)

# Formating x-axis ticks
plt.yticks(fontsize=12)
plt.xticks(fontsize=14)

fig.tight_layout()

plt.show()
```

Graph:



The Subplot Graphs show a strong correlation between the actual unemployment rate and the number of Google searches for the unemployment benefits, especially for the Covid-19 in 2020. The data clearly illustrates the immediate impact of the COVID-19 pandemic on both the U.S. labor market and public prosperity.

As the graph illustrates, the first effects of the pandemic became apparent in February 2020. By March 2020, the unemployment rate had peaked at about 15%, coinciding with a peak in Google search trend intensity at its relative maximum value of 100.

It is noteworthy that while the unemployment rate recovered more quickly after 2020 than it did previous the 2008 financial crisis, concerns in unemployment benefits remained at a high level, maintaining an average relative intensity of 60.

This persistently high search volume points to ongoing economic uncertainty, despite the macroeconomic recovery.

In conclusion, this analysis emphasizes the importance of Google Search Trends as a high-frequency, real-time indicator. It validates how search volume data can be effectively used by analysts and statisticians to assess public interest and quantify the immediate socioeconomic impacts of significant macroeconomic occurrences.

Conclusion:

In this project, we compared Google search trends with Tesla stock price, Bitcoin price, and unemployment rates in the United States to see whether search data reflects real socioeconomic behaviors. The results showed that Google Trends can be a useful supporting data source for socioeconomic analysis, although it should not be used alone.

The main focus of this report was applying data science techniques for analyzing time-series data using Python. We used `.describe()` to view descriptive statistics, `.resample()` to convert datasets to the same periodicity, and `.isna().values.sum()` to find missing values. We also worked with `matplotlib.dates` locators to format the time axis, changed chart resolution using the figure DPI. In addition, we improved our ability to style and customize visualizations by adjusting limits, labels, markers, colors, resolution, and by using grids to identify possible seasonality patterns.

As part of the project, we used pandas, matplotlib, and seaborn to analyze and visualize the datasets. We created dual-axis plots and stacked subplots to compare multiple variables simultaneously and formatted the time axis to make long-term trends easier to identify. As part of the analysis, we examined various approaches and applied them to different datasets to prepare, transform, and visualize time-series data within a clear and professional data science workflow.

References:

<https://www.udemy.com/course/100-days-of-code/>

https://pandas.pydata.org/docs/user_guide/index.html#user-guide

<https://matplotlib.org/stable/users/index.html>

<https://seaborn.pydata.org/tutorial.html>

<https://trends.google.com/home>

<https://www.rev.com/transcripts/tesla-cybertruck-event-transcript-elon-musk-unveils-cybertruck>

<https://www.blackwellglobal.co.uk/5-best-cryptocurrency-2019/>

<https://www.federalreservehistory.org/essays/great-recession-and-its-aftermath>